

标准数据结构:

如果一个 $(s(N), t(N))$ —实现满足以下条件, 那么称其为一个标准数据结构:

1. 当数据结构申请一个新内存块时, 首先选择若干已有块 $A_1, A_2, \dots, A_k$, 然后按顺序复制到新数据块中:
 1. 把 A_1 的元素复制到新内存块中;
 2. 释放 A_1 ;
 3. 把 A_2 的元素复制到新内存块中;
 4. 释放 A_2 ;
 5.
2. 复制完成后, 临时空间马上小时. 也就是说, $t(N)$ 的临时空间只在分配新块和复制元素时短暂使用.

Growth game:

为了能够证明标准实现 grow 操作摊还成本的下界, 论文第 8 章提出了一个 growth game, 具体定义如下:

对于 (N, k, l) —growth game, 给定三个非负整数 N, k, l , 其中:

- N : 游戏结束前总共要插入的元素数量;
- k : 可以使用的子数组数量;
- l : 一个新建或合并后的子数组中允许预留的空位数量.

游戏维护 k 个子数组 $A_1, A_2, \dots, A_k$, 它们共同表示一个逻辑上的可变长数组 A . 为了简化后续的合并, 我们使元素倒序排列, 即 A 最早的元素存放在 A_k 中, 接下来的存在 A_{k-1} , 以此类推. 令 $a_i = |A_i|$, 表示子数组 A_i 中当前的元素数量.

规定空的子数组必须在前面. 也就是说, $\forall i$ 使得 $a_i = 0$, 对于 $\forall j, 0 < j < i$, 都有 $a_j = 0$.

规定只有第一个非空子数组可以有预留的空位. 也就是说, 如果 A_i 是第一个非空子数组 (即它前面的所有子数组都是空数组 $\forall j, 0 < j < i, a_j = 0$), 那么 A_i 可以有至多 l 个未使用的位置, 而 $A_{i+1}, \dots, A_k$ 必须全部装满, 即 $\forall j, i < j \leq k, a_j$ 等于 A_j 的容量. 因此, 整个数据结构中由于未填满而产生的空位最多就是 l .

综上, 该数据结构所用的额外空间最多为 $k + l$.

在 growth game 中, 忽略分配和释放所需的成本, 只考虑元素写入, 因此成本就是元素写入的次数. 每次 grow 操作加入一个新元素, 这有三种情况:

1. 第一个非空子数组有空位
那么就直接把新元素放进它的第一个空位, 操作成本显然为 1.
2. 所有非空子数组都已经装满了, 但还有空的子数组
那么就找到最后一个空子数组, 也就是编号最大的空子数组, 给它分配一个容量为 $l + 1$ 的新子数组 (也就是最多可以预留的空位数量, 加上一个新元素的空间), 并把新元素放进它的第一个位置.
因为忽略内存分配的成本, 此处只写入了一个元素, 因此操作成本也是 1.
注意在前两种情况下, 玩家没有操作空间.
3. 所有 k 个子数组都已经装满了
此时没有空子数组, 也没有预留空位. 需要玩家来选择一个整数 $i \in [k]$, 然后分配一个容量为 $(l + 1) + \sum_{j=1}^i a_j$ (也就是最多可以预留的空位数量, 加上一个新元素的空间, 再加上前 i 个子数组的元素总数), 然后把子数组 $A_i, A_{i-1}, \dots, A_1$ 中的元素复制到新数组中, 复制时要保持逻辑数组的元素顺序. 接下来把新元素放入该数组的第一个空位, 此时新数组中刚好还剩下 l 个空位. 最后释放旧的 $A_1, \dots, A_i$, 让新数组成为新的 A_i . 最

后 $A_1, \dots, A_{i-1}$ 变为空, 而 $A_{i+1}, \dots, A_k$ 保持不变.

操作成本为 $1 + \sum_{j=1}^i a_j$, 即一个新元素的写入成本和所有被复制的旧元素的所需成本之和.

Growth game 的目标是: 找到完成一个 (N, k, l) -growth game 的全部 N 次插入所需要的最小总成本 $C_{N,k,l}$, 以及相应的摊还成本 $A_{N,k,l} = \frac{C_{N,k,l}}{N}$, 和实现这个最小成本的最佳移动序列.

如果一个真实数据结构有 $cN^{1/r}$ 的额外空间, 那么它可以对应一个 $k = cN^{1/r}, l = cN^{1/r}$ 的 growth game. 实际上, growth game 是比真实数据结构更宽松的一个模型, 因为:

- 玩家提前知道最终要插入的元素数量 N , 因此可以制定全局最优计划;
- 真实数据结构的空位限制与其当前数组大小有关, 例如当数组有 n 个元素时, 合理的额外空间限制是 $cn^{1/r}$, 而 growth game 允许在一开始就使用最大的 $cN^{1/r}$, 这使得 growth game 相比真实情况有更多可用的额外空间;
- 真实数据结构增长时通常需要暂时同时保存旧内存块、新内存块以及其他辅助信息, 使其需要更大的额外空间, 但 growth game 忽略了内存分配成本.

这些优势只会使 growth game 的最小成本更小, 比真实问题更容易, 所以 growth game 的最小成本可以充当真实数据结构最优解的一个下界.

$l = 0$ 情况的分析:

实际上, 空位的数量 l 对降低平均成本没有帮助. 举例来说, 假设 $l = 2$, 每次分配一个新的子数组后, 新的子数组内先放入一个新元素, 然后剩下 2 个空位. 接下来两个新的元素只会依次填入这两个空位而没有玩家的操作空间.

也就是说, 在玩家每次决策之后, 总会跟着 l 次强制操作. 因此我们实际上可以把连续的 $l + 1$ 次插入元素看作一次操作, 把这 $l + 1$ 个元素捆绑为 1 个元素. 如此, 元素写入所需的总成本缩小到了 $\frac{1}{l+1}$, 但元素数量也缩小到了 $\frac{1}{l+1}$, 因此平均成本不变, 即引理 8.2: 对于任意能被 $l + 1$ 整除的 N , $C_{N,k,l} = (l + 1)C_{\frac{N}{l+1},k,0}$ 且 $A_{N,k,l} = A_{\frac{N}{l+1},k,0}$.

这样一来, 我们就将 growth game $l > 0$ 的情况规约为了 $l = 0$ 的情况. 接下来主要研究 $l = 0$ 的情况. 记 $C_{N,k} = C_{N,k,0}$, $A_{N,k} = A_{N,k,0}$, (N, k) -growth game = $(N, k, 0)$ -growth game.

用向量 $\mathbf{a} = (a_1, a_2, \dots, a_k) \in \mathbb{N}^k$ 表示 (N, k) -growth game 的一个状态;

定义 $P_{N,k}$ 为 (N, k) -growth game 所有合法的最终状态的集合, 即 $P_{N,k} = \{\mathbf{a} \mid \sum_{i=1}^k a_i = N \text{ 且 } a_i = 0 \Rightarrow a_{i-1} = 0, i = 2, \dots, k\}$;

对于 $\mathbf{a} \in P_{N,k}$, 定义 $C(\mathbf{a}) = C_k(\mathbf{a})$ 为从状态 $(0, 0, \dots, 0)$ 到状态 $\mathbf{a}$ 所需的最小成本, 显然 $C_{N,k} = \min_{\mathbf{a} \in P_{N,k}} C_k(\mathbf{a})$.

(引理 8.4) 接下来我们关注每个子数组的形成过程, 从而把总成本拆开分析:

1. 假设最终状态是 $(a_1, a_2, \dots, a_{k-1}, a_k)$, 考虑操作过程中, 最后一次改变 A_k 的时刻. 改变 A_k 要么是将新元素填入 A_k , 此时前面的子数组均为空; 要么是将 $A_1, \dots, A_k$ 合并为 A_k , 填入新元素并清空 $A_1, \dots, A_{k-1}$; 第一种情况实际上可以归纳为第二种情况. 因此, 这次操作结束后的状态一定是 $(0, 0, \dots, 0, a_k)$. 在此之后, A_k 不再改变, 达到最终状态所需成本是建立前 $k - 1$ 个子数组所需的成本 $C_{k-1}(a_1, \dots, a_{k-1})$. 由此可得递推式 $C_k(a_1, a_2, \dots, a_k) = C_k(0, \dots, 0, a_k) + C_{k-1}(a_1, a_2, \dots, a_{k-1})$.
2. 由上式归纳可得 $C_k(a_1, a_2, \dots, a_k) = \sum_{j=1}^k C_j(0, \dots, 0, a_j)$.
3. 考虑状态 $(0, \dots, 0, a_k)$. 要达到这个状态所需的最后一步操作要求将此前已经存在的 $a_k - 1$ 个元素合并到新块中, 并写入新元素, 因此最后这步操作的成本为 a_k ; 而在这步之前, 要组织好已有的 $a_k - 1$ 个元素所需的最低

成本是 $C_{a_k-1,k}$.

因此 $C_k(0, \dots, 0, a_k) = C_{a_k-1,k} + a_k$.

4. 综上, 对于 $\mathbf{a} \in P_{N,k}$, 如果有 $0 = a_{i-1} < a_i$, 那么

$$C(\mathbf{a}) = \sum_{j=i}^k C_j(0, \dots, 0, a_j) = \sum_{j=i}^k C_{a_j-1,j} + a_j = N + \sum_{j=i}^k C_{a_j-1,j}.$$

由此我们把计算 $C(\mathbf{a})$ 拆成了计算若干个 $C_{N,k}$ 的子问题.

$C_{N,k}$ 的计算:

论文在定理 8.6 中给出了关于 $C_{N,k}$ 的以下命题:

1. 如果对某个 $n \geq 0$ 有 $\binom{n+k-1}{k} - 1 \leq N \leq \binom{n+k}{k} - 1$, 那么 $C_{N,k} = (N+1)n - \binom{n+k}{k+1}$;
2. 如果对某个 $n \geq 1$ 有 $\binom{n+k-1}{k} \leq N < \binom{n+k}{k}$, 那么 $C_{N,k} - C_{N-1,k} = n$;
3. 如果对某个 $n \geq 0$ 有 $\binom{n+k-1}{k} - 1 \leq N < \binom{n+k}{k} - 1$, 那么 $\forall i \in [k]$ 有 $C_{N,k} = C(\mathbf{a}) \Leftrightarrow \binom{n+i-2}{i} \leq a_i \leq \binom{n+i-1}{i}$;

可以通过数学归纳法证明以上定理. 为了证明以上定理, 我们需要以下命题:

(引理 8.5)

$$1. \binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$$

这是一个众所周知的公式.

$$2. \sum_{i=0}^k \binom{n+i-1}{i} = \binom{n+k}{k}$$

可由以下归纳步骤证得: $\sum_{i=0}^k \binom{n+i-1}{i} = (\sum_{i=0}^{k-1} \binom{n+i-1}{i}) + \binom{n+k-1}{k} = \binom{n+k-1}{k-1} + \binom{n+k-1}{k} = \binom{n+k}{k}$.

(命题 8.8) 当 $n = 1$, 即 $0 \leq N \leq k$ 时, $C_{N,k} = N$, 并且 $\forall \mathbf{a} \in P_{N,k}$, $C(\mathbf{a}) = N$ 当且仅当 $\forall i \in [k]$, $0 \leq a_i \leq 1$.

(命题 8.9) 假设定理 8.6(1) 对 (N, k) 和 $(N-1, k)$ 都成立, 那么: 如果对某个 $n \geq 1$ 有 $\binom{n+k-1}{k} \leq N < \binom{n+k}{k}$, 显然 $\binom{n+k-1}{k} - 1 < N \leq \binom{n+k}{k} - 1$ 且 $\binom{n+k-1}{k} - 1 \leq N-1 < \binom{n+k}{k} - 1$. 应用定理 1, 可得 $C_{N,k} - C_{N-1,k} = ((N+1)n - \binom{n+k}{k+1}) - (Nn - \binom{n+k}{k+1}) = n$. 也就是说, 当定理 8.6(1) 对 (N, k) 和 $(N-1, k)$ 都成立时, 定理 8.6(2) 对 (N, k) 成立.

(命题 8.10) 假设定理 8.6(1) 对所有满足 $N' < N$ 且 $k' \leq k$ 的 (N', k') 都成立, 那么取 $\mathbf{a} \in P_{N,k}$ 满足 $\forall i \in [k]$, $\binom{n+i-2}{i} \leq a_i \leq \binom{n+i-1}{i}$, 则

$$\begin{aligned}
C(a) &= N + \sum_{i=1}^k C_{a_i-1,i} \text{(引理8.4)} \\
&= N + \sum_{i=1}^k (a_i(n-1) - \binom{n+i-1}{i+1}) \text{(定理8.6(1))} \\
&= N + N(n-1) - \sum_{i=1}^k \binom{n+i-1}{i+1} \\
&= Nn - \sum_{i=1}^k \binom{n+i-1}{i+1} \\
&= (N+1)n - (1 + (n-1) + \sum_{i=1}^k \binom{n+i-1}{i+1}) \\
&= (N+1)n - \left(\binom{n-2}{0} + \binom{n-1}{1} + \sum_{j=2}^{k+1} \binom{n+j-2}{j} \right) \\
&= (N+1)n - \sum_{j=0}^{k+1} \binom{n+j-2}{j} \\
&= (N+1)n - \sum_{j=0}^{k+1} \binom{(n-1)+j-1}{j} \\
&= (N+1)n - \binom{(n-1)+(k+1)}{k+1} \text{(引理8.5)} \\
&= (N+1)n - \binom{n+k}{k+1}
\end{aligned}$$

综上, 当定理 8.6 (1) 对所有满足 $N' < N$ 且 $k' \leq k$ 的 (N', k') 都成立时, $\forall \mathbf{a} \in P_{N,k}$ 满足 $\forall i \in [k], \binom{n+i-2}{i} \leq a_i \leq \binom{n+i-1}{i}$, 有 $C(\mathbf{a}) = (N+1)n - \binom{n+k}{k+1}$.

(命题 8.11) 考虑对某个 $n \geq 1$ 有 $\binom{n+k-1}{k} - 1 \leq N < \binom{n+k}{k} - 1$, 取 $\mathbf{a} \in P_{N,k}$, 那么:

1. 假设 $\forall j \in [k], a_j \leq \binom{n+j-2}{j}$, 则
 $N = \sum_{j=1}^k a_j < \sum_{j=1}^k \binom{n+j-2}{j} = \sum_{j=0}^k \binom{(n-1)+j-1}{j} - 1 = \binom{n+k-1}{k} - 1$ (引理 8.5),
 与 $N \geq \binom{n+k-1}{k} - 1$ 矛盾.
 因此假设不成立, 如果 $\exists i \in [k]$ 使得 $a_i < \binom{n+i-2}{i}$, 那么一定 $\exists j \in [k]$ 使得 $a_j > \binom{n+j-2}{j}$.
2. 类似地, 假设 $\forall i \in [k], a_i \geq \binom{n+i-1}{i}$, 则
 $N = \sum_{i=1}^k a_i > \sum_{i=1}^k \binom{n+i-1}{i} = \sum_{i=0}^k \binom{n+i-1}{i} - 1 = \binom{n+k}{k} - 1$ (引理 8.5),
 与 $N \leq \binom{n+k}{k} - 1$ 矛盾.
 因此假设不成立, 如果 $\exists j \in [k]$ 使得 $a_j > \binom{n+j-1}{j}$, 那么一定 $\exists i \in [k]$ 使得 $a_i < \binom{n+i-1}{i}$.

(命题 8.12) 假设定理 8.6 (2) 对所有满足 $N' < N$ 且 $k' \leq k$ 的 (N', k') 都成立, 并且对某个 $n \geq 1$ 有 $\binom{n+k-1}{k} - 1 \leq N \leq \binom{n+k}{k} - 1$, 取 $\mathbf{a} \in P_{N,k}$, 但不满足 $\forall i \in [k], \binom{n+i-2}{i} \leq a_i \leq \binom{n+i-1}{i}$:

1. 如果 $\exists i \in [k]$ 使得 $a_i < \binom{n+i-2}{i}$, 根据命题 8.11 (1), $\exists j \in [k]$ 使得 $a_j > \binom{n+j-2}{j}$. 取 $\mathbf{b} = (b_1, b_2, \dots, b_k) \in P_{N,k}$, 其中 $b_i = a_i + 1, b_j = a_j - 1, \forall l \in [k] \setminus \{i, j\}$. 根据定理 8.6 (2), 有 $C_{a_i,i} - C_{a_i-1,i} \leq n-2$ 且 $C_{a_j-1,j} - C_{a_j-2,j} \geq n-1$. 因此

$$\begin{aligned}
C(\mathbf{b}) - C(\mathbf{a}) &= N + \sum_{l: b_l > 0} C_{b_l-1, l} - N - \sum_{l: a_l > 0} C_{a_l-1, l} \text{ (引理 8.4)} \\
&= C_{b_i-1, i} + C_{b_j-1, j} - C_{a_i-1, i} - C_{a_j-1, j} \\
&= C_{a_i, i} + C_{a_j-2, j} - C_{a_i-1, i} - C_{a_j-1, j} \\
&= (C_{a_i, i} - C_{a_i-1, i}) - (C_{a_j-1, j} - C_{a_j-2, j}) \\
&< 0
\end{aligned}$$

2. 如果 $\exists j \in [k]$ 使得 $a_j < \binom{n+j-1}{j}$, 根据命题 8.11 (1), $\exists i \in [k]$ 使得 $a_i > \binom{n+i-1}{i}$. 取 $\mathbf{b} = (b_1, b_2, \dots, b_k) \in P_{N, k}$, 其中 $b_i = a_i + 1, b_j = a_j - 1, \forall l \in [k] \setminus \{i, j\}$. 根据定理 8.6 (2), 有 $C_{a_i, i} - C_{a_i-1, i} \leq n - 1$ 且 $C_{a_j-1, j} - C_{a_j-2, j} \geq n$. 因此

$$\begin{aligned}
C(\mathbf{b}) - C(\mathbf{a}) &= N + \sum_{l: b_l > 0} C_{b_l-1, l} - N - \sum_{l: a_l > 0} C_{a_l-1, l} \text{ (引理 8.4)} \\
&= C_{b_i-1, i} + C_{b_j-1, j} - C_{a_i-1, i} - C_{a_j-1, j} \\
&= C_{a_i, i} + C_{a_j-2, j} - C_{a_i-1, i} - C_{a_j-1, j} \\
&= (C_{a_i, i} - C_{a_i-1, i}) - (C_{a_j-1, j} - C_{a_j-2, j}) \\
&< 0
\end{aligned}$$

因此如果定理 8.6 (2) 对所有满足 $N' < N$ 且 $k' \leq k$ 的 (N', k') 都成立, 并且对某个 $n \geq 1$ 有 $\binom{n+k-1}{k} - 1 \leq N \leq \binom{n+k}{k} - 1$, 取 $\mathbf{a} \in P_{N, k}$, 但不满足 $\forall i \in [k], \binom{n+i-2}{i} \leq a_i \leq \binom{n+i-1}{i}$, 那么一定存在 $\mathbf{b}$ 满足 $C(\mathbf{a}) > C(\mathbf{b})$, 所以 $C(\mathbf{a}) > C_{N, k}$, 即 $\mathbf{a}$ 一定不是一个最优的最终状态.

现在我们终于可以证明定理 8.6 了.

1. 由命题 8.8 可知, 定理在 $0 \leq N \leq k$ 时成立.

2. 我们需要证明如果定理对所有满足 $N' < N$ 且 $k' \leq k$ 的 (N', k') 都成立, 那么对 (N, k) 也成立. 由命题 8.10 可知, 所有满足 $\forall i \in [k], \binom{n+i-2}{i} \leq a_i \leq \binom{n+i-1}{i}$ 的 $\mathbf{a} \in P_{N, k}$ 都有 $C(\mathbf{a}) = (N+1)n - \binom{n+k}{k+1}$, 同时由命题 8.12 可知不满足上述条件的 $\mathbf{a}$ 一定不是一个最优的最终状态, 所以最小成本 $C_{N, k}$ 就是 $(N+1)n - \binom{n+k}{k+1}$. 定理 8.6 的 (1) (3) 得证. 又由命题 8.9 可知 (2) 成立.

定理 8.6 得证.

由定理 8.6, 我们可以得到以下推论:

(推论 8.13)

- 如果 $N = \binom{n+k}{k} - 1$, 那么根据定理 8.6 (1), $C_{N, k} = (N+1)n - \binom{n+k}{k+1} = \binom{n+k}{k}n - \binom{n+k}{k+1} = \binom{n+k}{k}n - \frac{n}{k+1} \binom{n+k}{k} = \frac{kn}{k+1} \binom{n+k}{k} = \frac{kn}{k+1} (N+1)$.
- 如果 $N = \binom{n+k-1}{k}$, 那么 $C_{N, k} > C_{N-1, k} = \frac{k(n-1)}{k+1} N$;
如果 $\binom{n+k-1}{k} < N \leq \binom{n+k}{k} - 1$, 那么根据定理 8.6 (2), $C_{N, k} = C_{N-1, k} + n \geq \frac{k(n-1)}{k+1} (N-1) + n \geq \frac{k}{k+1} (n-1) N$. 因此 $A_{N, k} \geq \frac{k}{k+1} (n-1) \geq \frac{1}{2} (n-1)$.

二项式计数器:

我们已经知道了如何计算最小成本; 现在我们需要解决: 怎么通过一步一步操作, 来达到这个最小成本.

(引理 8.14) 当对某个 $n \geq 0$ 有 $N = \binom{n+k}{k} - 1$ 时, 由引理 8.5 可知 $\sum_{i=1}^k \binom{n-i+1}{i} = \binom{n+k}{k} - 1 = N$, 因此 $\forall i \in [k], a_i = \binom{n-i+1}{i}$ 是一个最优的最终状态. 同时, 根据定理 8.6, 此时所有 a_i 都取到上界, 不存在某个 a_i 可以取更小的值. 因此该最终状态为唯一的最优最终状态. 根据引理 8.4, 要达到该状态, 所有中间状态也必须满足 $\forall i \in [j], a_i = \binom{n-i+1}{i}$, 因此要达到这一状态所需的操作序列也是唯一的.

所以, 当对某个 $n \geq 0$ 有 $N = \binom{n+k}{k} - 1$ 时, (N, k) -growth game 有唯一的最优操作序列.

$N = \binom{n+k}{k} - 1$ 的这一最优解可以用一个二项式计数器表达, 伪码如下:

```

1 Initialize(k):
2   a1, a2, ..., ak ← 0
3   b1, b2, ..., bk ← 0
4   bk+1 ← ∞

1 Increment(k):
2   i ← min{j ∈ [k] | bj < bj+1}
3   ai ← 1 + Σ(j=1..i) aj
4   bi ← bi + 1
5   a1, a2, ..., ai-1 ← 0
6   b1, b2, ..., bi-1 ← 0

```

用 $(b_1, b_2, \dots, b_k)$ 维护一个计数器, 初始化全部为 0. 增长时, 找到最小的满足 $b_i < b_{i+1}$ 的下标 i (形如 $b_1 = b_2 = \dots = b_i < b_{i+1}$), 将其加一, 并清空其前面的计数器项, 类似进位 (因为 b_{k+1} 被初始化为 ∞ , 所以总能找到满足条件的下标). 与此同时, 将 A 的前 i 个子数组合并到 A_i 并插入新元素, 伪码中体现为 $a_i \leftarrow 1 + \sum_{j=1}^i a_j$; $a_1, a_2, \dots, a_{i-1} \leftarrow 0$.

(引理 8.15) 下面证明二进制计数器的正确性. 我们要证明在使用二进制计数器时, $\forall i \in [k], a_i = \binom{b_i + i - 1}{i}$:

1. 初始状态下上式成立, 因为 $\forall i \in [k], 0 = \binom{0+i-1}{i}$.
2. 假设上式对当前 $(a_1, a_2, \dots, a_k)$ 和 $(b_1, b_2, \dots, b_k)$ 成立, 设 $b_1 = b_2 = \dots = b_i = n < b_{i+1}$.
新值 $a'_i = 1 + \sum_{j=1}^i a_j$, $b'_i = b_i + 1$.
$$a'_i = 1 + \sum_{j=1}^i a_j = 1 + \sum_{j=1}^i \binom{n+j-1}{j} = \sum_{j=0}^i \binom{n+j-1}{j} = \binom{n+i}{i} = \binom{b'_i + i - 1}{i}.$$

上式得证.

(引理 8.16) 考虑以下时刻: $b_1 = b_2 = \dots = b_k = n$, 根据引理 8.15, 此时 $N = \sum_{i=1}^k a_i = \sum_{i=1}^k \binom{n+i-1}{i} = \binom{n+k}{k} - 1$, 根据引理 8.14, 二项式计数器达到该状态的过程是 (N, k) -growth game 的唯一最优解.

Growth game 扩展规则:

前面的 growth game 要求, 合并子数组时必须把从 A_1 开始的所有子数组合并, 即对某个 $i \in [k]$, $(a_1, a_2, \dots, a_k) \rightarrow (0, \dots, 0, 1 + \sum_{l=1}^i a_l, a_{i+1}, \dots, a_k)$. 如果放宽这一限制, 采用更灵活的合并方式, 能否取得更低的成本?

1. 允许合并中间的连续子数组, 即对于 $1 \leq i < j \leq k$, $(a_1, a_2, \dots, a_k) \rightarrow (0, \dots, 0, a_1, \dots, a_{i-1}, \sum_{l=i}^j a_l, a_{j+1}, \dots, a_k)$, 也就是将 $A_i, A_{i+1}, \dots, A_j$ 合并为新的 A_j , 并把前面的非空数组重新编号来补上合并后的空位. (注意在这一过程中没有加入新元素.)
该操作的成本是 $\sum_{l=i}^j a_l$.
记 $(a_1, a_2, \dots, a_k)$ 为 $\mathbf{a}$
记 $(0, \dots, 0, a_1, \dots, a_{i-1}, \sum_{l=i}^j a_l, a_{j+1}, \dots, a_k)$ 为 $\mathbf{a}'$
(引理 8.17) 使用原规则, 要达到状态 $\mathbf{a}'$, 所需最小成本为 $C_k(\mathbf{a}')$; 先使用原规则到达状态 $\mathbf{a}$, 然后通过前述操作到达 $\mathbf{a}'$ 所需的最小成本为 $C_k(\mathbf{a}) + \sum_{l=i}^j a_l$.

$$C_j(a_1, a_2, \dots, a_j) + C_k(0, \dots, 0, a_{j+1}, \dots, a_k) = \sum_{l=1}^j C_l(0, \dots, 0, a_l) + \sum_{l=j+1}^k C_l(0, \dots, 0, a_l) = \sum_{l=1}^k C_l(0, \dots, 0, a_l) = C_k(a_1, \dots, a_k).$$

记 $S = C_k(0, \dots, 0, a_{j+1}, \dots, a_k)$

因此

$$\begin{aligned}
C_k(\mathbf{a}') &= C_j(0, \dots, 0, a_1, a_2, \dots, a_{i-1}, \sum_{l=i}^j a_l) + S \\
&= C_{j-1}(0, \dots, 0, a_1, a_2, \dots, a_{i-1}) + C_j(0, \dots, 0, \sum_{l=i}^j a_l) + S \\
&\leq C_{j-1}(0, \dots, 0, a_1, a_2, \dots, a_{i-1}) + C_j(0, \dots, 0, a_i, a_{i+1}, \dots, a_j - 1) + \sum_{l=i}^j a_l + S \\
&\leq C_{j-1}(0, \dots, 0, a_1, a_2, \dots, a_{i-1}) + C_j(0, \dots, 0, a_i, a_{i+1}, \dots, a_j) + \sum_{l=i}^j a_l + S \\
&\leq C_{i-1}(a_1, a_2, \dots, a_{i-1}) + C_j(0, \dots, 0, a_i, a_{i+1}, \dots, a_j) + \sum_{l=i}^j a_l + S \\
&= C_j(a_1, a_2, \dots, a_j) + \sum_{l=i}^j a_l + S \\
&= C_j(\mathbf{a}) + \sum_{l=i}^j a_l
\end{aligned}$$

综上所述的新操作对降低最小成本没有帮助。

2. 允许合并任意一组块. 取一个下标集合 $I = \{i_1, i_2, \dots, i_r\} \subseteq [k]$, 其中 $i_1 < i_2 < \dots < i_r$, 然后将 $A_{i_1}, A_{i_2}, \dots, A_{i_r}$ 合并到 A_{i_r} 并加入新元素, 也就是 $a_{i_r} \leftarrow 1 + \sum_{l=1}^r a_{i_l}$, 对所有 $l \in [r-1], a_{i_l} \leftarrow 0$. 将这种操作称为一个 l-move. 证明过程和引理 8.17 一致, l-move 对降低最小成本也没有帮助。

下界分析:

现在, 给定 N, k, l 的情况下, 我们可以得到 growth game 的最优解法。

假设 $r = O(\log N)$, 且一个标准实现的可变长数组可用的额外空间为 $O(rN^{1/r})$. 令额外空间为 $rN^{1/r}$, 忽略常数, 因为对于任意 $\alpha > 1$, 在 N 足够大时都有 $\alpha rN^{1/r} < (r-1)N^{1/(r-1)}$. 因为 growth game 使用的额外空间为 $k + l$, 所以 $k \leq rN^{1/r}, l \leq rN^{1/r}$, 放宽下界, 取 $k = l = rN^{1/r}$. 对这个可变长数组的任何操作都被扩展的 $(N, rN^{1/r}, rN^{1/r})$ -growth game 中的操作覆盖了, 这意味着真实数据结构的最小成本不会低于对应的, 条件更宽松的 growth game 的最小成本。

引理 8.2 已经证明了 $l > 0$ 的 growth game 可以规约为 $l = 0$ 的 growth game 而保持最小摊还成本不变. 在当前情况下, 对应的 $l = 0$ 的 growth game 是 $(\frac{N}{rN^{1/r}+1}, rN^{1/r}, 0)$ -growth game. 在 N 足够大时常数 1 可以忽略, 不会影响渐近结果, 因此可以改写为 $(\frac{1}{r}N^{1-1/r}, rN^{1/r})$ -growth game.

根据推论 8.13, 我们需要找到一个 n , 满足 $\binom{n+rN^{1/r}}{n} < \frac{1}{r}N^{1-1/r}$, 此时有摊还成本 $A_{N, rN^{1/r}, rN^{1/r}} = A_{\frac{1}{r}N^{1-1/r}, rN^{1/r}} \geq \frac{1}{2}n$. 只要能证明有 $n = \Theta(r)$, 就可以得到摊还成本下界 $A_{N, rN^{1/r}, rN^{1/r}} \geq \frac{n}{2} = \Omega(r)$.

把 $n = \Theta(r)$ 写成 $n = \beta r$, 其中 $\beta > 0$ 是一个常数. 应用不等式 $\binom{n}{k} \leq (\frac{en}{k})^k$ 可得: $\binom{n+rN^{1/r}}{n} = \binom{\beta r+rN^{1/r}}{\beta r} \leq \binom{(\beta+1)r+rN^{1/r}}{\beta r} \leq (\frac{e(\beta+1)N^{1/r}}{\beta})^{\beta r} = (\frac{e(\beta+1)}{\beta})^{\beta r} N^{\beta}$. 我们需要证明 $(\frac{e(\beta+1)}{\beta})^{\beta r} N^{\beta} < \frac{1}{r}N^{1-1/r}$

把 $r = O(\log N)$ 写成 $r \leq C \log N$, C 为常数.

要证 $(\frac{e(\beta+1)}{\beta})^{\beta r} N^{\beta} < \frac{1}{r} N^{1-1/r}$,

即证 $\log((\frac{e(\beta+1)}{\beta})^{\beta r} N^{\beta}) < \log(\frac{1}{r} N^{1-1/r})$,

即证 $\beta r \log(\frac{e(\beta+1)}{\beta}) + \beta \log N < (1 - \frac{1}{r}) \log N - \log r$,

即证 $\beta \frac{r}{\log N} \log(\frac{e(\beta+1)}{\beta}) + \beta < 1 - \frac{1}{r} - \frac{\log r}{\log N}$.

当 $\beta \rightarrow 0^+$ 时, $\beta \frac{r}{\log N} \log(\frac{e(\beta+1)}{\beta}) + \beta \leq \beta C \log(\frac{e(\beta+1)}{\beta}) + \beta \rightarrow 0$, 因此总能取到一个足够小的 β , 使 $(\frac{e(\beta+1)}{\beta})^{\beta r} N^{\beta} < \frac{1}{2}$;

另一方面, 当 $r \geq 3$ 且 N 足够大时, 因为 $r = O(\log N)$, 所以 $\frac{\log r}{\log N} \rightarrow 0$, 又 $1 - \frac{1}{r} \geq \frac{2}{3}$, 因此有 $1 - \frac{1}{r} - \frac{\log r}{\log N} > \frac{1}{2}$.

综上, 总能取到合适的 β 使得 $\beta \frac{r}{\log N} \log(\frac{e(\beta+1)}{\beta}) + \beta < 1 - \frac{1}{r} - \frac{\log r}{\log N}$ 成立, 即总能取到合适的 β 使得 $A_{N, rN^{1/r}, rN^{1/r}} = \Omega(r)$ 成立.

结合第 6 章的结论. 第 6 章所构造的实现, 有增长摊还成本 $O(r)$. 合并上下界, 有增长摊还成本 $\Theta(r)$. 显然这个实现达到了渐近最优.

结语:

论文提出了一类可变长数组的实现, 在以下方面实现了最优权衡:

- 存储数组所需的空间
- 调整数组大小时临时所需的空间
- 扩展和收缩操作的摊还成本

于此同时, 这些实现仍然保证访问操作的最坏情况成本为 $O(1)$.

扩展操作的摊还成本下界目前只适用与文中提出的标准算法 (即申请新内存块后, 把若干旧块里的元素按顺序复制到新块中, 然后释放旧块的算法), 但作者相信在元素和指针不可压缩的假设下, 这些下界可以推广到所有算法.

本文为了得到扩展操作摊还成本的下界, 定义了 growth game, 并进行了精确求解. 不过这一求解过程较为复杂, 作者认为是否存在一种更简单的求解方法是一个值得研究的问题.

作者认为本文的方法还可以用于改进 3.4 节描述的模型, 这是未来的研究项目.